

《软件安全》实验报告

姓名：谢小珂 学号：2310422 班级：1069

实验名称：

Angr 应用示例

实验要求：

根据课本 8.4.3 章节，复现 sym-write 示例的两种 angr 求解方法，并就如何使用 angr 以及怎么解决一些实际问题做一些探讨。

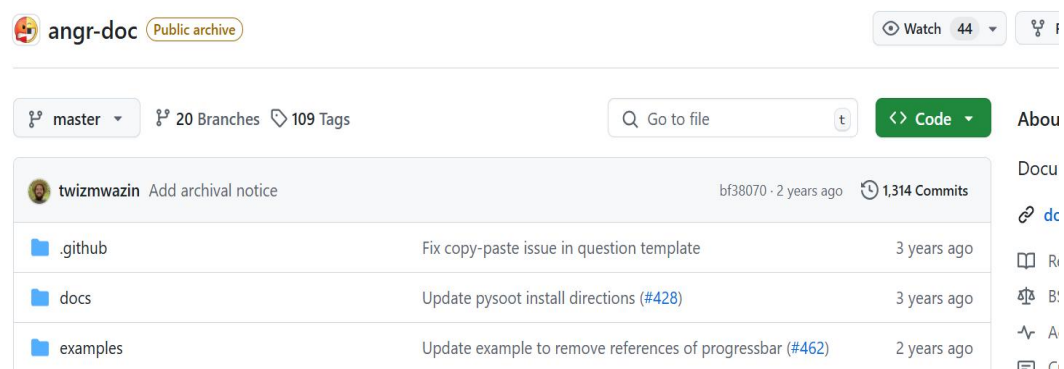
实验过程：

1. 安装 Angr

打开命令控制台，执行以下命令安装 Angr: `pip install angr`。

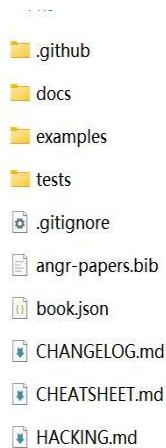
```
Downloading angr-9.2.154-py3-none-win_amd64.whl.metadata (5.0 kB)
Collecting cxxheaderparser (from angr)
  Downloading cxxheaderparser-1.5.0-py3-none-any.whl.metadata (5.2 kB)
Collecting GitPython (from angr)
  Downloading GitPython-3.1.44-py3-none-any.whl.metadata (13 kB)
Collecting ailment==9.2.154 (from angr)
  Downloading ailment-9.2.154-py3-none-any.whl.metadata (1.5 kB)
Collecting archinfo==9.2.154 (from angr)
  Downloading archinfo-9.2.154-py3-none-any.whl.metadata (1.8 kB)
Collecting cachetools (from angr)
  Downloading cachetools-5.5.2-py3-none-any.whl.metadata (5.4 kB)
Collecting capstone==5.0.3 (from angr)
```

安装完成后，对 Python 环境进行测试，先输入命令 `python`，进入 Python 交互界面。接着在该界面中输入代码 `import angr`，若运行后未报错且能正常执行，则表明 angr 库已成功安装，并且可以从 GitHub 上获取 angr 的官方文档。



2. angr 解法一：solve.py

在 examples 目录下定位到 sym-write 文件夹并将其打开，其中存在用于测试的代码文件 `issue.c`，具体内容如下：



```
1  #include <stdio.h>
2
3  char u=0;
4  int main(void)
5  {
6      int i, bits[2]={0,0};
7      for (i=0; i<8; i++) {
8          bits[(u&(1<<i))!=0]++;
9      }
10     if (bits[0]==bits[1]) {
11         printf("you win!");
12     }
13     else {
14         printf("you lose!");
15     }
16     return 0;
17 }
```

solve.py 文件部分源码如下:

```
20
21     u = claripy.BVS("u", 8)
22     state.memory.store(0x804a021, u)
23
24     sm = p.factory.simulation_manager(state)
25
26     def correct(state):
27         try:
28             return b'win' in state.posix.dumps(1)
29         except:
30             return False
31     def wrong(state):
32         try:
33             return b'lose' in state.posix.dumps(1)
34         except:
35             return False
36
37     sm.explore(find=correct, avoid=wrong)
38
39     # Alternatively, you can hardcode the addresses.
40     # sm.explore(find=0x80484e3, avoid=0x80484f5)
41
```

在这段代码里定义了名为 main 的函数。整个 Python 程序会执行 `print(repr(main()))` 语句，这一操作会调用 main 函数，接着借助 `repr()` 函数把 main 函数返回的对象转换为字符串形式，最后将该字符串打印输出。

Angr 执行分析的关键步骤如下:

1>工程初始化

使用 `angr.Project` 加载目标二进制文件 `./issue`, 并设置 `auto_load_libs=False` 以避免自动加载依赖库, 防止符号执行陷入库函数调用的复杂性。

2>状态初始化

使用 `p.factory.entry_state()` 创建一个初始状态对象 `state`, 该对象封装了程序执行所需的内存、寄存器、文件系统等环境信息。也可以通过 `blank_state(addr=0x1234)` 指定程序的起始执行地址。

3>变量符号化

使用 `claripy.BVS("u", 8)` 定义 8 位变量 `u`, 并通过 `state.memory.store(0x804a021, u)` 将其存入二进制文件的固定地址。

4>创建模拟管理器

通过 `p.factory.simulation_manager(state)` 创建一个模拟管理器 `sm`, 用于跟踪和管理

符号执行过程中产生的所有可能状态分支。

5>执行符号搜索

调用 `sm.explore(find=correct, avoid=wrong)` 进行符号执行，其中 `correct` 和 `wrong` 是用户定义的回调函数，用于判断当前状态是否满足预期条件。示例中通过检查标准输出是否包含 `win` 或 `lose` 字符串来判断执行结果。也可以直接指定地址（如 `find=0x80484e3` 和 `avoid=0x80484f5`），这些地址对应 IDA 反汇编中 `printf(you win!)` 和 `printf(you lose!)` 的指令位置。

6>约束求解

对符合条件的状态（`sm.found`），调用求解器获取符号变量 `u` 的值。

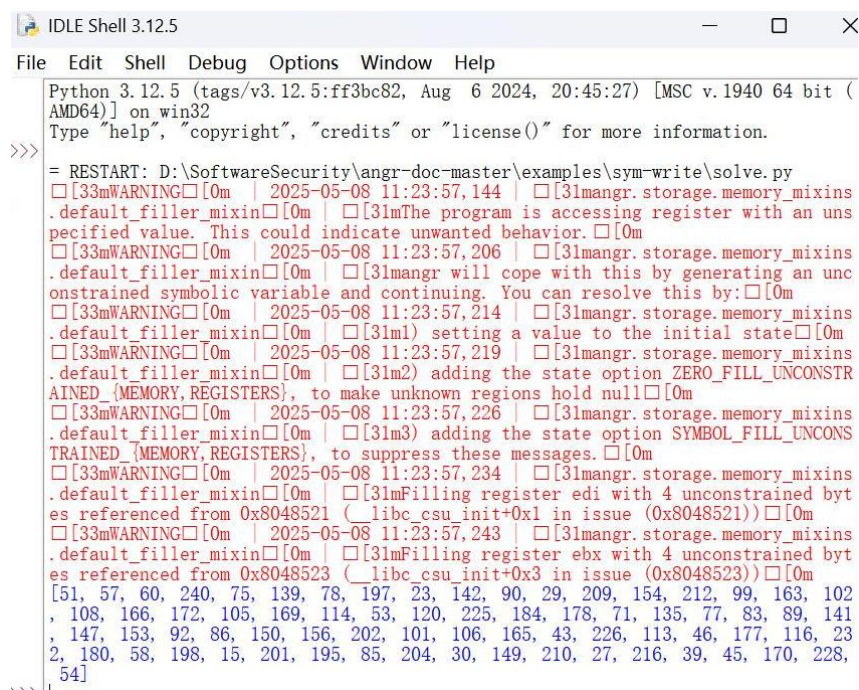
`eval_upto(e, n, cast_to=None, **kwargs)` 求解一个表达式多个可能的求解方案，`e` 表达式，`n` 所需解决方案的数量；

`eval(e, **kwargs)` 评估一个表达式以获得任何可能的解决方案；

`eval_one(e, **kwargs)` 求解表达式以获得唯一可能的解决方案。

实验结果

程序开发完成后，将 `solve.py` 文件拖曳至 Python IDLE 集成开发环境并执行，即可获得变量 `u` 的全部可行解：



```
IDLE Shell 3.12.5
File Edit Shell Debug Options Window Help
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:\SoftwareSecurity\angr-doc-master\examples\sym-write\solve.py
[33mWARNING[0m | 2025-05-08 11:23:57,144 | [31mangr.storage.memory_mixins
.default_filler_mixin[0m | [31mThe program is accessing register with an uns
pecified value. This could indicate unwanted behavior. [0m
[33mWARNING[0m | 2025-05-08 11:23:57,206 | [31mangr.storage.memory_mixins
.default_filler_mixin[0m | [31mangr will cope with this by generating an unc
onstrained symbolic variable and continuing. You can resolve this by: [0m
[33mWARNING[0m | 2025-05-08 11:23:57,214 | [31mangr.storage.memory_mixins
.default_filler_mixin[0m | [31ml) setting a value to the initial state [0m
[33mWARNING[0m | 2025-05-08 11:23:57,219 | [31mangr.storage.memory_mixins
.default_filler_mixin[0m | [31ml2) adding the state option ZERO_FILL_UNCONST
RAINED {MEMORY, REGISTERS}, to make unknown regions hold null [0m
[33mWARNING[0m | 2025-05-08 11:23:57,226 | [31mangr.storage.memory_mixins
.default_filler_mixin[0m | [31ml3) adding the state option SYMBOL_FILL_UNCONS
TRAINED {MEMORY, REGISTERS}, to suppress these messages. [0m
[33mWARNING[0m | 2025-05-08 11:23:57,234 | [31mangr.storage.memory_mixins
.default_filler_mixin[0m | [31mFilling register edi with 4 unconstrained byt
es referenced from 0x8048521 (__libc_csu_init+0x1 in issue (0x8048521)) [0m
[33mWARNING[0m | 2025-05-08 11:23:57,243 | [31mangr.storage.memory_mixins
.default_filler_mixin[0m | [31mFilling register ebx with 4 unconstrained byt
es referenced from 0x8048523 (__libc_csu_init+0x3 in issue (0x8048523)) [0m
[51, 57, 60, 240, 75, 139, 78, 197, 23, 142, 90, 29, 209, 154, 212, 99, 163, 102
, 108, 166, 172, 105, 169, 114, 53, 120, 225, 184, 178, 71, 135, 77, 83, 89, 141
, 147, 153, 92, 86, 150, 156, 202, 101, 106, 165, 43, 226, 113, 46, 177, 116, 23
2, 180, 58, 198, 15, 201, 195, 85, 204, 30, 149, 210, 27, 216, 39, 45, 170, 228,
54]
```

蓝色部分，就是输出的 `u` 的求解的结果，因为我们采用了 `eval_upto_` 函数，给出了多个解，对每个解我们都可以带入源程序进行验证。

3. angr 解法二：solve2.py

新建 `solve2.py` 文件写入如下代码：

```

1 import angr
2 import claripy
3 def hook_demo(state):
4     state.regs.eax=0
5 p= angr.Project("./issue",load_options={"auto_load_libs": False})
6 p.hook(addr=0x08048485,hook=hook_demo,length=2)
7 state = p.factory.blank_state(addr=0x0804846B,
8                               add_options={"SYMBOLIC WRITE ADDRESSES"})
9 u= claripy.BVs("u",8)
10 state.memory.store(0x804A021, u)
11 sm= p.factory.simulation_manager(state)
12 sm.explore(find=8x980484DB)
13 st = sm.found[e]
14 print(repr(st.solver.eval(u)))

```

比较与 solve.py 的区别

1>采用 Hook 函数优化（关键改进）

solve2.py 在地址 0x08048485 处通过 p.hook() 替换了原始指令,改用自定义函数直接设置寄存器值:

```

def hook_demo(state):
    state.regs.eax = 0 # 功能等价于 xor eax, eax

```

p.hook(0x08048485, hook_demo, length=2)

避免模拟执行冗余指令,显著提升符号执行速度。此方法可扩展至复杂函数的替换,减少路径爆炸风险。

2>符号执行条件的变化

solve.py: 需同时定义 find 和 avoid 条件,或指定两个地址 (0x80484e3 和 0x80484f5)。

solve2.py: 因 win/lose 分支互斥,仅需 find=0x080484DB,无需 avoid,减少约束条件,加速符号执行收敛。

3>结果求解方式变化

solve.py: 使用 eval_upto(u, 256) 枚举所有可能解,适用于需全面分析的场景。

solve2.py: 改用 eval(u) 返回单个解,适用于仅需一个可行解的快速验证场景。

实验结果

```

IDLE Shell 3.12.5
File Edit Shell Debug Options Window Help
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: D:\SoftwareSecurity\angr-doc-master\examples\sym-write\solve2.py
[33mWARNING[0m | 2025-05-08 14:11:16,986 | [31mangr.storage.memory_mixins.default_filler_mixin[0m | [31mThe program is accessing memory with an unspecified value. This could indicate unwanted behavior.[0m
[33mWARNING[0m | 2025-05-08 14:11:17,014 | [31mangr.storage.memory_mixins.default_filler_mixin[0m | [31mangr will cope with this by generating an unconstrained symbolic variable and continuing. You can resolve this by:[0m
[33mWARNING[0m | 2025-05-08 14:11:17,029 | [31mangr.storage.memory_mixins.default_filler_mixin[0m | [31m1) setting a value to the initial state[0m
[33mWARNING[0m | 2025-05-08 14:11:17,039 | [31mangr.storage.memory_mixins.default_filler_mixin[0m | [31m2) adding the state option ZERO_FILL_UNCONSTRAINED_MEMORY, REGISTERS, to make unknown regions hold null[0m
[33mWARNING[0m | 2025-05-08 14:11:17,059 | [31mangr.storage.memory_mixins.default_filler_mixin[0m | [31m3) adding the state option SYMBOL_FILL_UNCONSTRAINED_MEMORY, REGISTERS, to suppress these messages.[0m
[33mWARNING[0m | 2025-05-08 14:11:17,072 | [31mangr.storage.memory_mixins.default_filler_mixin[0m | [31mFilling memory at 0x7fff0000 with 4 unconstrained bytes referenced from 0x8048472 (main+0x7 in issue (0x8048472))[0m
[33mWARNING[0m | 2025-05-08 14:11:17,089 | [31mangr.storage.memory_mixins.default_filler_mixin[0m | [31mFilling register ebp with 4 unconstrained bytes referenced from 0x8048475 (main+0xa in issue (0x8048475))[0m
226
>>>

```


心得体会：

在本次实验里，我成功复现了 `sym - write` 示例中的两种 `angr` 求解方法，在此过程中，对 `angr` 的运用方式以及如何借助它解决实际问题展开了深入探究。借助 `hook` 函数，有效提升了符号执行的性能，同时对符号执行的不同状态条件进行了探索，最终成功求解出符号变量，切实验证了 `angr` 在二进制分析领域的强大效能。

一、Angr 的使用方法

1. `angr` 库安装：`angr` 库的安装操作简便，利用 `pip` 包管理器，执行“`pip install angr`”命令就能完成安装。不过，针对某些特定的依赖项，有时还需要额外配置。

2. 加载二进制文件：借助 `angr.Project` 类可以实现对目标二进制文件的加载。通过设置 `auto_load_libs` 选项，能够灵活控制是否自动加载二进制文件所依赖的库文件。

3. 构建模拟管理器：先创建 `SimState` 对象来代表模拟程序状态，再运用 `p.factory.simulation_manager(state)` 构建模拟管理器 `sm`，由它负责管理模拟执行过程中产生的各类状态。

4. 执行分析操作：使用 `sm.explore` 方法对程序开展符号执行工作，执行时需要明确指定要查找的目标状态（`find`）以及要避免的状态（`avoid`）。

5. 提取分析结果：完成符号执行后，借助求解器 `st.solver.eval` 提取符号变量的值，以便后续对结果进行分析和处理。

二、Angr 可解决的实际问题

1. 漏洞挖掘：`angr` 凭借符号执行和约束求解技术，能够自动探寻二进制程序所有可能的执行路径，助力发现像缓冲区溢出、格式化字符串漏洞、整数溢出等潜在的安全漏洞，这对增强软件的安全性和稳定性意义重大。

2. 逆向工程：即便没有源代码，`angr` 也可用于剖析二进制程序的行为。这对于研究闭源软件或恶意软件的行为极为有用，能为安全研究人员和逆向工程师的深入分析提供有力支持。

3. 自动化测试：`angr` 可以生成用于覆盖程序不同执行路径的测试用例，进而提升软件测试的覆盖率。这种自动化测试手段有助于发现程序中隐藏的错误，提高软件质量。

4. 程序理解：`angr` 提供了丰富的分析工具，能协助开发者更好地理解程序的执行流程、控制流图以及数据依赖关系等。借助这些工具，开发者在进行代码审查和优化时会更加高效。

5. 二进制补丁：通过分析，`angr` 能帮助开发者找出并理解漏洞产生的根本原因，从而开发出相应的补丁。对于存在已知漏洞的二进制程序，`angr` 可实现问题定位并辅助修复。

6. 游戏破解：虽然游戏破解并非 `angr` 的主要应用方向，但它可以用于分析游戏程序，查找游戏内的逻辑漏洞或薄弱环节，以创建作弊工具或游戏修改器。利用符号执行，能够自动找出游戏中的漏洞。

7. 安全评估：安全研究人员可运用 `angr` 对软件进行安全评估，识别并修复潜在的安全问题。通过全面的路径探索和约束求解，`angr` 能够提供详尽的安全评估报告。

8. 竞态条件分析：`angr` 能够检测多线程程序中的竞态条件，帮助开发者找出并修复可能引发不确定行为的问题，这对于保障多线程应用的稳定性和可靠性至关重要。

9. 定制分析：鉴于 `angr` 具备良好的可扩展性，开发者可依据实际需求编写自定义的分析插件，解决特定问题。`angr` 的插件机制为开发者扩展其功能、满足特定需求提供了便利。

angr 强大的符号执行和约束求解能力，极大地提高了漏洞挖掘、逆向工程、自动化测试、程序理解等任务的效率。持续不断地实践和探索，有助于更深入地掌握 angr 的使用技巧，并将其灵活应用于实际问题的解决当中。